

SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

COMPARATIVE ANALYSIS

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

QUESTIONS:

1. Coin Change vs Dice Throw
2. Dijkstra vs Prim's Algorithm
3. Kruskal vs Boruvka Algorithm

1. Coin Change vs Dice Throw

Aim:

To compare an optimization DP problem with a counting DP problem.

Given: Coins = [1, 3, 4], Amount = 6; Dice = 2, Target sum = 6.

Concept:

Coin Change finds the minimum number of coins. Dice Throw counts the number of ways to obtain a target sum.

Python Code

```
def coin_change(coins, amount):
    dp = [999] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a-c] + 1)
    return dp[amount]

def dice_ways(dice, target):
    dp = [[0]*(target+1) for _ in range(dice+1)]
    dp[0][0] = 1
    for d in range(1, dice+1):
        for s in range(1, target+1):
            for face in range(1, 7):
                if face <= s:
                    dp[d][s] += dp[d-1][s-face]
    return dp[dice][target]

print(coin_change([1,3,4], 6))
print(dice_ways(2, 6))
```

Input: [1,3,4], amount = 6; 2 dice, sum = 6

Output: Minimum coins = 2; Number of dice ways = 5

Result:

6 = 3 + 3, so the minimum is 2 coins. For two dice, the five ordered combinations give sum 6.

```
main.py
1 def coin_change(coins, amount):
2     dp = [999] * (amount + 1)
3     dp[0] = 0
4     for a in range(1, amount + 1):
5         for c in coins:
6             if c <= a:
7                 dp[a] = min(dp[a], dp[a-c] + 1)
8     return dp[amount]
9
10 def dice_ways(dice, target):
11     dp = [[0]*(target+1) for _ in range(dice+1)]
12     dp[0][0] = 1
13     for d in range(1, dice+1):
14         for s in range(1, target+1):
15             for face in range(1, 7):
16                 if face <= s:
17                     dp[d][s] += dp[d-1][s-face]
18     return dp[dice][target]
19
20 print(coin_change([1,3,4], 6))
21 print(dice_ways(2, 6))
```

2
5

2. Dijkstra vs Prim's Algorithm

Aim:

To compare shortest-path calculation using Dijkstra with MST construction using Prim's algorithm.

Given edges: A-B:2, A-C:3, B-C:1, B-D:4, C-D:5.

Concept:

Dijkstra selects the nearest unvisited vertex to find shortest paths. Prim selects the minimum edge that expands the MST.

Python Code

```
import heapq

g = {
    'A': [('B', 2), ('C', 3)], 'B': [('A', 2), ('C', 1), ('D', 4)],
    'C': [('A', 3), ('B', 1), ('D', 5)], 'D': [('B', 4), ('C', 5)]
}

def dijkstra(g, src):
    dist = {v: float('inf') for v in g}
    dist[src] = 0
    q = [(0, src)]
    while q:
        d, u = heapq.heappop(q)
        if d != dist[u]: continue
        for v, w in g[u]:
            if d+w < dist[v]:
                dist[v] = d+w
                heapq.heappush(q, (dist[v], v))
    return dist

print(dijkstra(g, 'A'))
# Prim selects: A-B(2), B-C(1), B-D(4)
```

Input: Graph given above; source/start vertex = A

Output: Dijkstra distances = A:0, B:2, C:3, D:6; Prim MST = A-B, B-C, B-D; total weight = 7

Result:

Dijkstra gives shortest paths from A, while Prim produces an MST. Both are greedy but have different objectives.

main.py

```
1 import heapq
2
3 g = {
4     'A': [('B', 2), ('C', 3)], 'B': [('A', 2), ('C', 1), ('D', 4)],
5     'C': [('A', 3), ('B', 1), ('D', 5)], 'D': [('B', 4), ('C', 5)]
6 }
7
8 def dijkstra(g, src):
9     dist = {v: float('inf') for v in g}
10    dist[src] = 0
11    q = [(0, src)]
12    while q:
13        d, u = heapq.heappop(q)
14        if d != dist[u]: continue
15        for v, w in g[u]:
16            if d+w < dist[v]:
17                dist[v] = d+w
18                heapq.heappush(q, (dist[v], v))
19    return dist
20
21 print(dijkstra(g, 'A'))
22 # Prim selects: A-B(2), B-C(1), B-D(4) |
```

✓ ↗ 🏠 ⚙️ 📁
{ 'A': 0, 'B': 2, 'C': 3, 'D': 6 }

...Program finished with exit code 0
Press ENTER to exit console.

3. Kruskal vs Boruvka Algorithm

Aim:

To compare two greedy algorithms for constructing a Minimum Spanning Tree.

Given edges: A-B:1, B-C:4, C-D:2, D-A:3, B-D:5.

Concept:

Kruskal sorts edges and selects safe edges. Boruvka selects the cheapest outgoing edge from each component.

Python Code

```
edges = [('A', 'B', 1), ('B', 'C', 4), ('C', 'D', 2),
         ('D', 'A', 3), ('B', 'D', 5)]

# Kruskal: sort and select non-cycle edges
edges.sort(key=lambda x: x[2])
print("Kruskal:", edges[:3])

# For this graph, Boruvka selects the same MST edges:
print("Boruvka: A-B(1), C-D(2), D-A(3)")
```

Input: Graph edges given above.

Output: Kruskal MST = A-B, C-D, D-A; Boruvka MST = A-B, C-D, D-A; Total weight = 6

Result:

Both algorithms produce the same MST with weight 6, but their edge-selection strategies are different.

Comparison & Complexity

Algorithm	Purpose / Strategy	Result	Complexity
Coin Change	DP – minimum coins	2 coins	$O(nA)$
Dice Throw	DP – count ways	5 ways	$O(dS \cdot 6)$
Dijkstra	Greedy – shortest path	A:0 B:2 C:3 D:6	$O(E \log V)$
Prim	Greedy – MST	Weight 7	$O(E \log V)$
Kruskal	Greedy – sorted edges	Weight 6	$O(E \log E)$
Boruvka	Greedy – cheapest component edges	Weight 6	$O(E \log V)$

Overall Result

The experiments show that the correct algorithm depends on the problem objective. Dynamic Programming is used for Coin Change and Dice Throw, while Greedy methods are used for shortest paths and MST construction. The assigned examples demonstrate the difference in selection strategy, output, and complexity.

main.py

```
1 edges = [('A','B',1),('B','C',4),('C','D',2),
2         | ('D','A',3),('B','D',5)]
3
4 # Kruskal: sort and select non-cycle edges
5 edges.sort(key=lambda x: x[2])
6 print("Kruskal:", edges[:3])
7
8 # For this graph, Boruvka selects the same MST edges:
9 print("Boruvka: A-B(1), C-D(2), D-A(3)")
```



```
Kruskal: [('A', 'B', 1), ('C', 'D', 2), ('D', 'A', 3)]
Boruvka: A-B(1), C-D(2), D-A(3)
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```